

Allow Templates in Local Classes

Document #: D1988R0
Date: 2020-01-13
Project: Programming Language C++
EWG
Reply-to: Steve Downey
<sdowney@gmail.com, sdowney2@bloomberg.net>

§ 1 Abstract

Local classes should be permitted to have member templates.

§ 2 Before / After Table

Before	After
<pre>// Extracted From LLVM test case temp.mem/p2.cpp void fun() { struct foo { template <typename> struct bar {}; // Error template <typename> void baz() {} // Error template <typename> using corge = int; // Error }; }</pre>	<pre>// Extracted From LLVM test case temp.mem/p2.cpp void fun() { struct foo { template <typename> struct bar {}; // Allowed template <typename> void baz() {} // Allowed template <typename> using corge = int; // Allowed }; }</pre>

§ 3 Implementation Experience For C++98

Implemented in clang by deleting the check for template inside local class.

```
diff --git a/clang/lib/Sema/SemaTemplate.cpp b/clang/lib/Sema/SemaTemplate.cpp
index 5633582d679..d155bf8b5e4 100644
--- a/clang/lib/Sema/SemaTemplate.cpp
+++ b/clang/lib/Sema/SemaTemplate.cpp
@@ -7385,12 +7385,7 @@ Sema::CheckTemplateDeclScope(Scope *S, TemplateParameterList
    *TemplateParams) {
    if (CXXRecordDecl *RD = dyn_cast<CXXRecordDecl>(Ctx)) {
        // C++ [temp.mem]p2:
        // A local class shall not have member templates.
-       if (RD->isLocalClass())
-           return Diag(TemplateParams->getTemplateLoc(),
-                       diag::err_template_inside_local_class)
-               << TemplateParams->getSourceRange();
-       else
-           return false;
+       return false;
    }
```

```
}
}
```

Built a stage2 bootstrap of llvm, including libc++, which succeeded.

```
CXX=clang++-8 CC=clang-8 cmake -DCLANG_ENABLE_BOOTSTRAP=On \
-DCLANG_BOOTSTRAP_PASSTHROUGH=\
"CMAKE_INSTALL_PREFIX;CMAKE_BUILD_TYPE;LLVM_USE_LINKER;LLVM_PARALLEL_LINK_JOBS;LLVM_ENABLE_PROJECTS" \
\
-DCMAKE_INSTALL_PREFIX=~/install/llvm-localclass/ -DLLVM_ENABLE_LIBCXX=yes \
-DCMAKE_BUILD_TYPE=Release -DLLVM_ENABLE_ASSERTIONS=yes -DLLVM_USE_LINKER=lld \
-DLLVM_PARALLEL_LINK_JOBS=1 \
-DLLVM_ENABLE_PROJECTS=\
"clang;clang-tools-extra;compiler-rt;debuginfo-
tests;libcxx;libcxxabi;libunwind;lld;lldb;llvm" \
-G Ninja ../llvm-project/llvm/
```

The test suite produced one new set of errors, the tests checking that templates not be declared inside of a local class.

```
error: 'error' diagnostics expected but not seen:
File <snip>/temp/temp.decls/temp.mem/p2.cpp Line 8: templates cannot be declared inside
of a local class
File <snip>/temp/temp.decls/temp.mem/p2.cpp Line 9: templates cannot be declared inside
of a local class
File <snip>/temp/temp.decls/temp.mem/p2.cpp Line 10: templates cannot be declared
inside of a local class
File <snip>/temp/temp.decls/temp.mem/p2.cpp Line 11: templates cannot be declared
inside of a local class
File <snip>/temp/temp.decls/temp.mem/p2.cpp Line 12: templates cannot be declared
inside of a local class
5 errors generated.
```

§ 4 Why C++ 98

In C++98 there is no way for a local class to escape the scope in which it is declared. (Un)Fortunately in C++11 we allowed the auto keyword and the deduction of the return type of a function.

What this means for templates in local classes is that the instantiation point may be outside the scope that the class is declared in. This leads to complications.

If the local class is returned from such a function, and the member template is used, the context defining the member, in existing implementations is lost.

```
template<typename T> auto f(T t) {
    struct X {
        template<typename U> void g() {
            decltype(t) x = 123; }
    };
    return X();
}

void h() {
    f(nullptr).g<int>();
}
```

With the patch above, clang will ICE. The corresponding lambda expression is diagnosed as an error.

```
~~~C++ template auto f(T t) { auto y = [] (U u) { decltype(t) x = 123; }; return y; }
```

```
void h1() { auto k = f(nullptr); }; ~~~ clang reports: ~~~ :3:48: error: cannot initialize a variable of type
'decltype(t)' (aka 'nullptr_t') with an rvalue of type 'int'
```

```
auto y = [<typename U>(U u) { decltype(t) x = 123;};
```

^ ~~~

:7:22: note: in instantiation of function template specialization 'f' requested here

```
void h1() { auto k = f(nullptr);};
```

^

1 error generated.

Compiler returned: 1 ~~~

[Compiler Explorer](#)

Interestingly, there is some divergence in detecting the error. GCC doesn't unless k is invoked: ~~~C++ void h2() { auto k = f(nullptr); k(1);};~~~

```
<source>: In instantiation of 'f(T) [with T = std::nullptr_t::<lambda(U)> [with U = int]':
```

```
<source>:8:37:   required from here
```

```
<source>:3:52: error: cannot convert 'int' to 'std::nullptr_t' in initialization
```

```
3 |     auto y = [<typename U>(U u) { decltype(t) x = 123;};
  |                                     ^~~
  |                                     |
  |                                     int
```

Compiler returned: 1

[Compiler Explorer](#)

I believe that clang is being too aggressive in instantiation of the operator() of the lambda, but I have not constructed an example that errors on otherwise well-formed code.

§ 5 Modules and Reachability

The problem of unnamable types whose members might be templates is similar to that of types in modules that are not exported but are present in interfaces that are. ~~~C++ export module M; struct X {};

export { X func(){return X{}}; } ~~~ The name X is unavailable outside the module, but the function func is. auto x = func(); is valid code.

This suggests we could leverage Reachability in order to provide meaning to local classes outside their declarative region. From [module.reach](#)

pnum{3} A declaration D is reachable if, for any point P in the instantiation context ([module.context]),

pnum{(3.1)} D appears prior to P in the same translation unit, or

pnum{(3.2)} D is not discarded ([module.global.frag]), appears in a translation unit that is reachable from P, and either does not appear within a private-module-fragment or appears in a private-module-fragment of the module containing P.

[Note: Whether a declaration is exported has no bearing on whether it is reachable. — end note]

The language already requires that the declaration of D, the local class, appear before any use of it, because of the existing rules for auto.

`module.global.frag` provides

pnum{4} A declaration D in a global module fragment of a module unit is discarded if D is not decl-reachable from any top-level-declaration in the top-level-declaration-seq of the translation unit. [Note: A discarded declaration is neither reachable nor visible to name lookup outside the module unit, nor in template instantiations whose points of instantiation ([temp.point]) are outside the module unit, even when the instantiation context ([module.context]) includes the module unit. — end note]

If we extend decl-reachable to include local classes that are made available via the return type of its declaring function, the definition of the local class would be _{reachable} for purposes of instantiating its member functions.

I do not know if this is viable in any sense as an implementation, but it seems to be a viable direction in terms of the standard.

§ 6 Wording

(temp.mem.2) ~~A local class of non-closure type shall not have member templates.~~ Access control rules apply to member template names. A destructor shall not be a member template. A non-template member function ([dcl.fct]) with a given name and type and a member function template of the same name, which could be used to generate a specialization of the same type, can both be declared in a class. When both exist, a use of that name and type refers to the non-template member unless an explicit template argument list is supplied.

I have not figured out how to word decl-reachable yet.